Phase E: Reflection Skill — TDD Spec

> *Model: sonnet This is the core product feature. Every task here matters. Directory: packages/skill/*

E1: SKILL.md + Proactive Recall

Goal: SKILL.md < 500 tokens, instructions only. Proactive recall checks past mistakes before task execution.

Tests:

```
// E1.test.ts
// Test 1: SKILL.md is under 500 tokens
const content = readFileSync('packages/skill/SKILL.md', 'utf8')
const tokens = estimateTokens(content)
expect(tokens).toBeLessThan(500)

// Test 2: SKILL.md contains reflection trigger instructions
expect(content).toContain('mistakes.md')
expect(content).toContain('lessons.md')
expect(content).toContain('why did I think I was right')

// Test 3: SKILL.md contains external_experience delimiter instruction
expect(content).toContain('<external_experience>')

// Test 4: Proactive recall matches task description against local index
const matches = await proactiveRecall('directory restructure cross-repo')
expect(matches.length).toBeGreaterThan(0)
expect(matches[0].file).toBe('mistakes.md')
expect(matches[0].content).toContain('import paths')

// Test 5: Proactive recall returns empty for unrelated task
const noMatches = await proactiveRecall('write a poem about the sea')
expect(noMatches.length).toBe(0)

// Test 6: SKILL-GUIDE.md exists separately (for humans, not loaded into
context)
expect(existsSync('packages/skill/SKILL-GUIDE.md')).toBe(true)
const guide = readFileSync('packages/skill/SKILL-GUIDE.md', 'utf8')
expect(estimateTokens(guide)).toBeGreaterThan(500)  // guide can be long
```

E2: Install Script + Directory Setup

Goal: One command installs everything. Zero user decisions. Idempotent.

Tests:

```
// E2.test.ts
// Test 1: Creates reflection directory structure
await runInstall({ workspaceDir: testDir })
expect(existsSync(join(testDir, 'reflection/mistakes.md'))).toBe(true)
expect(existsSync(join(testDir, 'reflection/lessons.md'))).toBe(true)
expect(existsSync(join(testDir, 'reflection/feelings.md'))).toBe(true)
expect(existsSync(join(testDir, 'reflection/thoughts.md'))).toBe(true)
expect(existsSync(join(testDir, 'drafts'))).toBe(true)
expect(existsSync(join(testDir, 'published'))).toBe(true)

// Test 2: Appends to AGENTS.md without breaking existing content
const original = '# My Agent\n\nExisting content here.'
writeFileSync(join(testDir, 'AGENTS.md'), original)
await runInstall({ workspaceDir: testDir })
const after = readFileSync(join(testDir, 'AGENTS.md'), 'utf8')
expect(after).toContain('Existing content here.')
expect(after).toContain('AgentXP Skill')

// Test 3: Idempotent — running twice does not duplicate AGENTS.md block
await runInstall({ workspaceDir: testDir })
await runInstall({ workspaceDir: testDir })
const count = (readFileSync(join(testDir, 'AGENTS.md'),
'utf8').match(/AgentXP Skill/g) || []).length
expect(count).toBe(1)

// Test 4: reflection/ added to .gitignore automatically
await runInstall({ workspaceDir: testDir })
const gitignore = readFileSync(join(testDir, '.gitignore'), 'utf8')
expect(gitignore).toContain('reflection/')

// Test 5: config.yaml has only 3 human-readable fields
await runInstall({ workspaceDir: testDir })
const config = readFileSync(join(testDir, 'skills/agentxp/config.yaml'),
'utf8')
expect(config).toContain('agent_name')
expect(config).toContain('relay_url')
expect(config).toContain('visibility_default')
expect(config).not.toContain('privateKey')
expect(config).not.toContain('publicKey')
```

E2b: Identity Initialization + CLI Shim

Tests:

```
// E2b.test.ts
// Test 1: Keys generated to ~/.agentxp/identity/ on first install
await runInstall({ workspaceDir: testDir, homeDir: testHome })
expect(existsSync(join(testHome,
'.agentxp/identity/operator.key'))).toBe(true)
expect(existsSync(join(testHome,
'.agentxp/identity/operator.pub'))).toBe(true)

// Test 2: Keys are NOT generated on second install (idempotent)
const firstKey = readFileSync(join(testHome,
'.agentxp/identity/operator.pub'), 'utf8')
await runInstall({ workspaceDir: testDir, homeDir: testHome })
const secondKey = readFileSync(join(testHome,
'.agentxp/identity/operator.pub'), 'utf8')
expect(firstKey).toBe(secondKey)

// Test 3: agentxp CLI symlink created in PATH
await runInstall({ workspaceDir: testDir, homeDir: testHome })
const whichResult = execSync('which agentxp').toString().trim()
expect(whichResult).toBeTruthy()

// Test 4: agentxp status command works after install
const status = execSync('agentxp status --json').toString()
const parsed = JSON.parse(status)
expect(parsed.agent_name).toBeDefined()
expect(parsed.relay_connected).toBeDefined()
```

E3: Heartbeat Continuity

Tests:

```
// E3.test.ts
// Test 1: heartbeat-chain.md updated after session
await writeHeartbeatChain('session 1 did X and discovered Y')
const chain = readFileSync(reflectionDir + '/heartbeat-chain.md', 'utf8')
expect(chain).toContain('session 1 did X')

// Test 2: Hard cap at 800 tokens — overflow auto-compresses oldest entry
const longEntry = 'a'.repeat(3000)  // very long entry
await appendHeartbeatChain(longEntry)
const after = readFileSync(reflectionDir + '/heartbeat-chain.md', 'utf8')
expect(estimateTokens(after)).toBeLessThanOrEqual(800)

// Test 3: Compressed entry is a 1-sentence summary, not truncated
// (summary should be meaningful, not just cut off mid-word)
```

```
const summary = extractOldestEntry(after)
expect(summary.endsWith('.')).toBe(true)
expect(summary.length).toBeGreaterThan(20)
expect(summary.length).toBeLessThan(200)
```

E4: Rule-Based Reflection Parser

Tests:

```
// E4.test.ts
// Test 1: Parses structured entry correctly
const entry = `## 2026-04-11 Missed import paths
- Tried: reorganized directory, updated paths in main repo
- Expected: tests would pass
- Outcome: failed
- Learned: cross-repo operations require listing all affected imports
- Tags: refactoring, imports`

const parsed = parseReflectionEntry(entry)
expect(parsed.tried).toContain('reorganized directory')
expect(parsed.outcome).toBe('failed')
expect(parsed.learned).toContain('cross-repo')
expect(parsed.tags).toContain('refactoring')

// Test 2: Quality gate — too short → unparseable
const short = `## 2026-04-11 learned to be careful\n- Tried: did stuff\n-
Outcome: ok\n- Learned: be careful`
const result = parseReflectionEntry(short)
expect(result.publishable).toBe(false)
expect(result.reason).toContain('too short')

// Test 3: Quality gate — no specifics → unparseable
const vague = `## 2026-04-11 general lesson\n- Tried: some approach I tried
yesterday\n- Outcome: partial\n- Learned: this approach sometimes works and
sometimes doesn't`
const result2 = parseReflectionEntry(vague)
expect(result2.publishable).toBe(false)

// Test 4: Quality gate — specific content passes
const specific = `## 2026-04-11 Docker DNS fix\n- Tried: modified
/etc/resolv.conf and restarted container\n- Outcome: succeeded\n- Learned:
docker container DNS cache cleared on restart, not on config reload\n-
Tags: docker, networking, dns`
const result3 = parseReflectionEntry(specific)
expect(result3.publishable).toBe(true)
```

```
// Test 5: Unparseable entries moved to drafts/unparseable/
await processReflectionFile(join(testDir, 'reflection/mistakes.md'))
if (hasUnparseableEntries) {
  expect(existsSync(join(testDir, 'drafts/unparseable/'))).toBe(true)
}
```

E5: Periodic Distillation + LLM Demand Trigger

Tests:

```
// E5.test.ts
// Test 1: Distillation compresses old entries into core insights
await seedReflectionFile(100) // 100 entries
await runDistillation()
const distilled = readFileSync(join(testDir, 'reflection/mistakes.md'),
'utf8')
expect(estimateTokens(distilled)).toBeLessThan(2000)  // compressed

// Test 2: Archive created for raw entries
expect(existsSync(join(testDir, 'reflection/archive/'))).toBe(true)

// Test 3: LLM trigger fires when > 5 unparseable entries
await seedUnparseable(3)
await checkLLMTrigger()
expect(llmWasCalled).toBe(false)  // not yet

await seedUnparseable(3)  // now 6 total
await checkLLMTrigger()
expect(llmWasCalled).toBe(true)

// Test 4: LLM trigger does NOT fire on schedule (no fixed cron)
// Verify there is no fixed-schedule LLM call in the codebase
const cronConfig = readFileSync('packages/skill/src/distiller.ts', 'utf8')
expect(cronConfig).not.toContain('0 14 * * *')  // no fixed time trigger
```

E6: Batch Publish with Retry Queue

Tests:

```
// E6.test.ts
// Test 1: Publishable drafts sent to relay
await createDraft({ what: 'Docker DNS', tried: '...', outcome: 'succeeded',
learned: '...' })
await runBatchPublish(relayUrl)
```

```
const published = readdir(join(testDir, 'published'))
expect(published.length).toBe(1)

// Test 2: Failed publish stays in drafts with retry metadata
await createDraft(validEntry)
await runBatchPublish('wss://unreachable-relay.invalid')
const draft = readDraftFile()
expect(draft.retry_count).toBe(1)
expect(draft.last_attempt).toBeDefined()

// Test 3: Retry backoff doubles each time (15min → 30min → 60min cap)
expect(getNextRetryDelay(1)).toBe(15 * 60 * 1000)
expect(getNextRetryDelay(2)).toBe(30 * 60 * 1000)
expect(getNextRetryDelay(3)).toBe(60 * 60 * 1000)
expect(getNextRetryDelay(10)).toBe(60 * 60 * 1000)  // capped

// Test 4: Successfully published draft moved to published/ with relay
confirmation ID
await createDraft(validEntry)
await runBatchPublish(relayUrl)
const publishedFile = readPublishedFile()
expect(publishedFile.relay_event_id).toBeDefined()
expect(existsSync(draftPath)).toBe(false)  // removed from drafts

// Test 5: Pulse events pulled after publish
const pulses = await runBatchPublish(relayUrl)
expect(pulses).toBeDefined()  // pulse pull happened
```

E7: Local Experience Search

Tests:

```
// E7.test.ts
// Test 1: Keyword search finds matching entries
await seedReflectionFiles(['Docker DNS fix', 'Kubernetes networking',
'Python import error'])
const results = await localSearch('docker')
expect(results.length).toBeGreaterThan(0)
expect(results[0].title).toContain('Docker')

// Test 2: Summary returned by default (low token cost)
const results = await localSearch('docker')
expect(results[0].title).toBeDefined()
expect(results[0].outcome).toBeDefined()
expect(results[0].tags).toBeDefined()
```

```
expect(results[0].full_content).toBeUndefined()  // not in default response

// Test 3: Full content returned only when requested
const expanded = await localSearch('docker', { expand: results[0].id })
expect(expanded[0].full_content).toBeDefined()

// Test 4: Zero network calls (purely local)
const networkCallsBefore = getNetworkCallCount()
await localSearch('docker')
const networkCallsAfter = getNetworkCallCount()
expect(networkCallsAfter).toBe(networkCallsBefore)
```

E8: Local Server + Auto-Auth

Tests:

```
// E8.test.ts
// Test 1: Server starts and dashboard opens
const server = await startLocalServer()
expect(server.port).toBeGreaterThan(1024)
expect(server.port).toBeLessThan(65535)

// Test 2: Port is random each session (not fixed)
const server1 = await startLocalServer()
const server2 = await startLocalServer()
// Not guaranteed different but must not be hardcoded
expect(typeof server1.port).toBe('number')

// Test 3: Relay proxy auto-signs requests with local keys
const res = await
fetch(`http://localhost:${server.port}/api/v1/experiences`)
// Should not return 401 — auto-authenticated
expect(res.status).not.toBe(401)

// Test 4: SSRF prevention — private IP rejected as relay URL
const result = await validateRelayUrl('wss://192.168.1.100')
expect(result.valid).toBe(false)
expect(result.reason).toContain('private IP')

// Test 5: SSRF prevention — localhost rejected as relay URL
const result2 = await validateRelayUrl('wss://localhost:3141')
expect(result2.valid).toBe(false)

// Test 6: Valid relay URL accepted
```

```
const result3 = await validateRelayUrl('wss://relay.agentxp.io')
expect(result3.valid).toBe(true)
```

E9: Agent Sub-Key Auto-Renewer

Tests:

```
// E9.test.ts
// Test 1: Key with > 14 days remaining not renewed
const futureKey = { ...agentKey, expiresAt: Date.now()/1000 + 30 * 86400 }
await checkAndRenew(futureKey, operatorKey)
expect(renewWasCalled).toBe(false)

// Test 2: Key with < 14 days remaining is renewed
const soonKey = { ...agentKey, expiresAt: Date.now()/1000 + 10 * 86400 }
await checkAndRenew(soonKey, operatorKey)
expect(renewWasCalled).toBe(true)

// Test 3: New key has 90 day TTL from renewal date
const newKey = await renewKey(soonKey, operatorKey)
const expectedExpiry = Math.floor(Date.now()/1000) + 90 * 86400
expect(newKey.expiresAt).toBeCloseTo(expectedExpiry, -3)

// Test 4: Renewal is silent — no user notification needed
// (user never sees key management)
expect(renewalRequiresUserAction).toBe(false)
```

Phase E spec complete. Phase F-I spec next.